

System Skills A02 Report

Waranyu Kittikamet (6780676)

Assignment 2 tasks us with building a finance tracker program utilizing a dynamic linked list structure in C. The following will be an explanation of my finance tracker program. I will be describing how I implemented a linked list structure and its advantages, as well as each source file and how they work.

Linked List Structure

My linked list structure follows a circular linked list structure, by having the “head” of a linked list node connect to the “tail” of the linked list. Essentially, when we are the first node of a linked list and we try to go back, before the head, we end up at the end of the linked list, the tail. Choosing this type of linked list will enable us to more easily traverse the list, making the max walking time half the length of the list, since if we want to access a node that is near the end of the list, we can easily get to it faster by traversing in reverse.

Source Files

linkedlist.h

```
GNU nano 7.2 linkedlist.h
#ifndef LINKEDLIST_H
#define LINKEDLIST_H

// Defining the LinkedList Node structure;
typedef struct Node {
    struct Node *head;
    long amnt;
    char *desc;
    char *status;
    struct Node *tail;
} Node;

// Functions used for LinkedList Nodes
void addlast(Node **list, long value, char *words, int *length, long *total, char *stat);
void add(Node **list, long value, char *words, int pos, int *length, long *total);
void dlt(Node **list, int pos, int *length, long *total);
void clear_memory(Node **list);
void print(Node *list, int *length);

#endif
```

Our node struct is defined as having five variables:

1. **Node *head**: the connecting point of the node for the node that comes before it.
2. **long amnt**: the amount of currency that is being transacted. We choose long to support bigger values.
3. **char *desc**: the description of the payment. We use a char pointer so that the size allocation can happen dynamically
4. **char *status**: the status of the transaction (new, saved, ++ i, --d).
5. **struct Node *tail**: the connecting point of the node for the node that comes after it.

There are also five functions

1. **void addlast**: adds a node to the end of the linked list, as well as updates the total currency and length of the list.
2. **void add**: adds a node to a specific position in the linked list, as well as update the total currency and length of the list.
3. **void dlt**: marks a node at a specific position for deletion, as well as updating the total currency.
4. **void clear_memory**: clears all memory allocation for the list.
5. **void print**: prints all transactions (description, amount, and status).

We use a double pointer (Node **list) so that we can get access to the main pointer (list) so that updating is possible, instead of copying it into the function. The **print** function doesn't need it since all it needs to do is read and print.

llfunc.c

```
void addlast(Node **list, long value, char *words, int *length, long *total, char *stat){
    // Initialize the new transaction.
    Node *n = (Node*)malloc(sizeof(Node));
    if (n == NULL){
        printf("\nNode memory allocation failed\n");
        printf("\nFailed to add transaction\n");
        return;
    }
    n->amnt = value;
    // Allocate space for transaction description. The amount of memory corresponds to how long the
description is.
    n->desc = (char*)malloc(strlen(words) + 1);
    strcpy(n->desc, words);
    // Allocate space for the status of the added transaction.
    n->status = (char*)malloc(strlen(stat) + 1);
    strcpy(n->status, stat);

    if (*list == NULL){
        // In the case of an empty list, initialize the first node by having its head and tail
point to each other.
        n->tail = n;
        n->head = n;
        *list = n;
    }
    else {
        // Get both the first and last nodes.
        Node *first = *list;
        Node *last = first->head;

        // Set the last node's tail and the first node's head pointing to new transaction
        n->tail = first;
        n->head = last;
        last->tail = n;
        first->head = n;
    }
    if (value >= (long)0){
        printf("Income added.");
    }
    else {
        printf("Expense added.");
    }
    // Update the total number of currency after transactions and the length of the list.
    *total += value;
    (*length)++;
}
```

Explanation:

This function creates a new node and attaches it to the end of our circular linked list, essentially updating it. This new node contains the amount, description and status of the transaction as specified in the parameters. In addition, the function also updates the total transaction amount and length of the linked list.

The function first initializes a new Node pointer and allocates space, size of our Node datatype, for it using **malloc**. It then checks if the memory allocation succeeded by confirming if our new Node pointer **n*** is a NULL pointer. If it's not then we go ahead and assign its amount (**amnt**), description (**desc**) and status (**status**). With **desc** and **status** specifically, we use string pointers, therefore necessitating memory allocation. Once that is done, we connect the head

and tail of `n*` either to each other if the list is empty, or to the last and first node respectively. Finally, we inform the user of the income/expense added, and then increment the total transaction amount and length of the linked list.

```
void add(Node **list, long value, char *words, int pos, int *length, long *total){
    // Condition for an out of bounds position.
    if (pos > *length || pos < 0){
        printf("\nPosition is out of bounds\n");
        printf("\nFailed to add transaction\n");
        return;
    }
    // Initialize the new transaction.
    Node *n = (Node*)malloc(sizeof(Node));
    if (n == NULL){
        printf("\nNode memory allocation failed\n");
        printf("\nFailed to add transaction\n");
        return;
    }
    n->amnt = value;
    // Allocate space for the description and status of the transaction
    n->desc = (char*)malloc(strlen(words) + 1);
    strcpy(n->desc, words);
    n->status = (char*)malloc(strlen("+++ i") + 1);
    strcpy(n->status, "+++ i");
    Node *current = *list;

    // Find the node at the designated position.
    if (pos > *length/2){
        current = current->head;
        for (int i = 0; i < *length-pos; i++){
            current = current->head;
        }
    }
    else {
        for (int i = 0; i < pos; i++){
            current = current->tail;
        }
    }

    // Connect the ends.
    Node *prevNode = current->head;
    n->head = prevNode;
    n->tail = current;
    prevNode->tail = n;
    current->head = n;

    if (pos == 0) {
        *list = n;
    }

    if (value >= (long)0){
        printf("Income added at position %d.", pos);
    }
    else {
        printf("Expense added at position %d.", pos);
    }
    // Update the total number of currency after transactions and the length of the list.
```

```

    *total += value;
    (*length)++;
}

```

Explanation:

This function creates a new node and attaches it to the user specified position in the linked list. All the processes used in **addlast** are also used here. There are two exceptions. This function checks if the given position is out of the bounds of the list (less than 0 or more than the current length of the linked list). The Node traversal is also different. If the given position is more than half the length of the list, then list traversal will go in reverse (tail to head). Otherwise, go forward (head to tail).

```

void dlt(Node **list, int pos, int *length, long *total){
    // Check if the given position is out of bounds of the length of the list.
    if (pos >= *length || pos < 0){
        printf("\nPosition is out of bounds\n");
        printf("\nFailed to mark transaction for deletion\n");
        return;
    }
    if (*list == NULL){
        printf("\nNo transactions exists to delete\n");
        printf("\nFailed to mark transaction for deletion\n");
        return;
    }
    Node *current = *list;
    /*
        For a position that is past half way the length of the list, then we will traverse the
list from the tail to the head
        Otherwise, traverse normally from the head to the tail
    */
    if (pos > *length/2){
        for (int i = 0; i < *length-pos; i++){
            current = current->head;
        }
    }
    else {
        for (int i = 0; i < pos; i++){
            current = current->tail;
        }
    }

    free(current->status);
    current->status = (char*)malloc(strlen("--- d") + 1);
    strcpy(current->status, "--- d");
    *total -= current->amnt;
    printf("Transaction at position %d marked for deletion.", pos);
    return;
}

```

Explanation:

This function marks a user specified transaction for deletion using its position. The Node traversal and out-of-bounds check are all lifted from **add**. Once the function reaches the Node, it changes its status to “--- d” to mark it for deletion.

```

void clear_memory(Node **list){
    if (*list == NULL || list == NULL){ return; }
}

```

```

Node *current = *list;
Node *nextNode;
current->head->tail = NULL;

while (current != NULL){
    nextNode = current->tail;
    free(current->desc);
    free(current->status);
    free(current);
    current = nextNode;
}

*list = NULL;
printf("\nMemory cleared\n");
}

```

Explanation:

This function frees all the memory allocated for the list, its nodes, and their respective information. It does so by going Node by Node, each time freeing the memory in the current node before freeing the node itself and going to the next node. This is repeated until all nodes are freed, then the list gets turned into a NULL pointer.

```

void print(Node *list, int *length){
    if (list == NULL){
        printf("[ Transactions ]\n(Empty)\n");
        return;
    }
    Node *current = list;
    printf("[ Transactions ]\n");
    for (int i = 0; i < *length; i++){
        printf("%d. %s\t%ld\t%s\n", i+1, current->desc, current->amnt, current->status);
        current = current->tail;
    }
}

```

Explanation:

This function prints the description, amount and status of all the transactions. It does so in the following format:

```

[ Transactions ]
1. [desc] [amnt] [status]
2. [desc] [amnt] [status]
3. [desc] [amnt] [status]

```

main.c

```

// ---Variable Definition---
char conf[200]; // conf will be used to read the user input
long t = 0; // t is the total amount of currency left after all transactions (positive amount
being within budget, negative amount being over budger)
long *total = &t; // *total will be the pointer to t. We will be using a pointer in our
functions so that we can update t.
int l = 0; // l is the length of the circular, dynamic linked list.
int *length = &l; // *length will be the pointer to l. Same purpose as *total
char desc[100]; // desc will hold a description for a transaction until stored in a linked list
node.
long amnt; // amnt will hold the transaction amount until stored in a linked list node.
char status[10]; // status will hold the status of the transaction until stored in a linked list
node.

```

```
Node *list = NULL; // *list will be are linked list. Initialize as a NULL pointer until we
finish some pre-requisite checks
```

Explanation:

These are some of the variables we will need to initialize at the very start of the program running. This is because of processes, like pending user input and reloading a previous session, that go before the main process of the fintrack program (adding/deleting income/expenses).

```
// Section 1: Asking for user input (reloading previous session)
while(1) {
    printf("\nWould you like to resume your previous session? (y/n): ");
    // Read the line of text from input(stdin)
    if (fgets(conf, sizeof(conf), stdin) != NULL) {
        // Replace the newline with a null terminator for the if-conditions
        conf[strcspn(conf, "\n")] = '\0';
        if (strcmp(conf, "y") == 0 || strcmp(conf, "n") == 0) {
            break;
        }
        else {
            printf("\nConfirmation not recognized\n");
        }
    }
}
```

Explanation:

This section is responsible for asking the user whether they want to reload a previous session or not. The while loop using **while(1)** will keep whatever is running in the loop running infinitely, since the looping condition is constant, thus keeping pending the user for a confirmation. **fgets(conf, sizeof(conf), stdin)** then takes the user input (**stdin**) of the same size as the variable **conf**. If **conf** is not NULL, then we will proceed to check if the input is either “y” or “n”. In order to do so, we need to replace the newline in the user input with a NULL terminator so that we can properly compare it to another string. We use **strcspn(conf, "\n")** to find the position of the newline, then we pass it to **conf[strcspn(conf, "\n")] = '\0'**; to replace the newline. After that we use **strcmp** to compare the user input to our allowed terms. If they match, then we exit the loop.

```
// Section 2: Reloading previous transaction_log.txt
/*
    Insert all contents from the transaction_log.txt into the list
    If transaction_log.txt is empty or doesn't exist, start with an empty list.
*/
if (strcmp(conf, "y") == 0){
    printf("\nResuming from last session...\n\n");
    FILE *log = fopen("../logs/transaction_log.txt", "r");
    // Check if transaction_log.txt exists in the logs folder
    if (log != NULL){
        // conf is used to store the user inputted line
        while (fgets(conf, sizeof(conf), log) != NULL) {
            int p = sscanf(conf, "%s\t%ld\t%s", desc, &amnt, status);
            // If the transaction_log.txt has an entry that does not follow our
saving pattern, then we exit the program
            // Saving pattern: desc[tab space]amnt[tab space]status
            if (p != 3){
                printf("Previous session is corrupted");
                clear_memory(&list);
                return 1;
            }
        }
    }
}
```

```

        status[strcspn(status, "\n")] = '\0';
        addlast(&list, amnt, desc, length, total, status);
        printf("\n");
    }
    printf("\nLast session loaded.\n");
    fclose(log);
}
else {
    printf("\nNo previous transactions found. Continuing with new transaction.\n");
    fclose(log);
}
}

```

Explanation:

This section is responsible for resuming a previous session of fintrack if the user requests for it. If we get a “y” from the previous section, then we proceed to open transaction_log.txt in read mode using **FILE *log = fopen("../logs/transaction_log.txt", "r");**. If the file does not exist, we will proceed anyways to the next section. If the file does exist, then we read the contents of transaction_log.txt line by line, and add the contents each to our linked list using **addlast** to preserve the order in transaction_log.txt. We use **fgets(conf, sizeof(conf), log)** to get each line and **int p = sscanf(conf, "%s\t%ld\t%s", desc, &amnt, status);**. If one of the entries does not follow our format (p != 3, p tells us how many arguments follow the correct format and we are asking for three arguments), then we exit the program. Otherwise, we replace the newline in status, since it is the last argument in each line in transaction_log.txt, and then we add all the saved information into the linked list. Repeat until every item in transaction_log.txt has been loaded. Finally we close transaction_log.txt using **close(log)**.

```

// Section 3: Transaction Management
printf("\nAvailable actions:\nadd income [amount] [description]\nadd income [amount]
[description] [position]\n");
printf("add expense [amount] [description]\nadd expense [amount] [description]
[position]\ndelete [position]\nprint\nquit\n");
printf("Disclaimer: [position] starts from 1\n");
while(1) {
    int wscount=0; // wscount is our counter for whitespace in the user input
    char cmd[10]; // cmd will represent the first argument given in the user input
    char cmd2[10]; // cmd2 will represent the second argument given in the user input
    int pos; // pos will hold the user inputted position
    int p; // p will be used for sscanf and tell us how many arguments in the user input
    match the specified pattern
    printf("\nCurrent total: %ld\n", *total);
    if (*total >= (long)0){
        printf("Budget Status: Within Budget\n");
    }
    else {
        printf("Budget Status: Over Budget!\n");
    }
    printf("```\n\n```\n");
    // This part takes the user inputted line of arguments
    printf("\n> ");
    if (fgets(conf, sizeof(conf), stdin) != NULL) {
        // Replace the newline with a NULL terminator so the string can be used in
        comparisons
        conf[strcspn(conf, "\n")] = '\0';
        if (strcmp(conf, "quit") == 0) {
            break;
        }
    }
}

```

```

// Counting the whitespace between arguments helps indicate what command the
user inputted
for(int i = 0; i < strlen(conf); i++){
    if (conf[i] == ' '){ wscount++; }
    if (wscount == 4){ break; }
}
switch(wscount){
    // if command == "add [income/expense] [amount] [desc] [pos]" ->
add(&list, [amnt/-amnt], desc, pos-1, length, total);
    // Four whitespaces indicate adding an income/expense at a certain
position
    case 4:
        p = sscanf(conf, "%s %s %ld %s %d", cmnd, cmnd2, &amnt, desc,
&pos);
        if (p == 5 && strcmp(cmnd, "add") == 0){
            // Negative numbers are not allowed to be inputted as a
transaction
            if (amnt < (long)0){
                printf("\nInvalid transaction amount\n");
            }
            else if (strcmp(cmnd2, "income") == 0){
                add(&list, amnt, desc, pos-1, length, total);
                printf("\n");
            }
            else if (strcmp(cmnd2, "expense") == 0){
                add(&list, -amnt, desc, pos-1, length, total);
                printf("\n");
            }
            else {
                printf("\nInvalid Command\n");
            }
        }
        else {
            printf("\nInvalid Command\n");
        }
        break;
    // if command == "add [income/expense] [amount] [desc]" ->
addlast(&list, [amnt/-amnt], desc, length, total, "(new)");
    // Three whitespaces indicate adding an income/expense with no location
given
    case 3:
        p = sscanf(conf, "%s %s %ld %s", cmnd, cmnd2, &amnt, desc);
        if (p == 4 && strcmp(cmnd, "add") == 0){
            if (amnt < (long)0){
                printf("\nInvalid transaction amount\n");
            }
            else if (strcmp(cmnd2, "income") == 0){
                addlast(&list, amnt, desc, length, total,
"(new)");
                printf("\n");
            }
            else if (strcmp(cmnd2, "expense") == 0){
                addlast(&list, -amnt, desc, length, total,
"(new)");
                printf("\n");
            }
            else {

```



```

                                printf("\nInvalid Command\n");
                            }
                        }
                    else {
                        printf("\nInvalid Command\n");
                    }
                    break;
                // if command == "delete [pos]" -> dlt(&list, pos-1, length, total);
                // One whitespace indicate deleting a transaction at a certain position
                case 1:
                    p = sscanf(conf, "%s %d", cmd, &pos);
                    if (p == 2 && strcmp(cmd, "delete") == 0){
                        dlt(&list, pos-1, length, total);
                        printf("\n");
                    }
                    else{
                        printf("\nInvalid Command\n");
                    }
                    break;
                // if command == "print" -> print(list, length);
                // No whitespace indicates print
                default:
                    if (strcmp(conf, "print") == 0){
                        print(list, length);
                    }
                    else {
                        printf("\nInvalid Command\n");
                    }
                    break;
            }
        }
    }
}

```

Explanation:

This section handles the process of adding/deleting incomes/expenses and/or print the current list of transactions. First we use a **while(1)** loop to keep pending for user input, then we get the user input using **fgets(conf, sizeof(conf), stdin)**. We then clean up the user input using **conf[strcspn(conf, "\n")] = '\0'**; and then we count the whitespaces in the user input. The amount of whitespaces can clue us into what command the user wants to input. For example, "add income 300 gift 3" has 4 whitespaces in between the arguments, telling us that it's an add command specifying a position. We use **switch(wscount)** to indicate and direct all our whitespace cases. Case 4 is for adding an income/expense at a specific position. Case 3 is for adding an income/expense at the end of the list. Case 1 is for deleting a transaction at a specific position. And default is for print. Each case has conditions for handling incorrect/unrecognized commands including checking **p** for the correct arguments and using **strcmp** to compare **cmd2** to "income"/"expense". The addition/insertion commands also have an additional condition where if the inputted amount is negative, it will not add/insert the transaction. Finally, the user can type "quit" to exit this section and proceed with saving the current transactions.

```

// Section 4: Saving and Clearing
// Save all transactions into ../logs/transaction_log.txt, clear memory and exit the program.
printf("\nSaving transactions to file...\n\n");
FILE *log = fopen("../logs/transaction_log.txt", "w");
if (log == NULL){
    printf("\nUnable to save\n");
    clear_memory(&list);
    return 1;
}

```

```

    }
    if (list != NULL) {
        printf("# Format: TYPE|DESCRIPTION|AMOUNT\n# TYPE: INC (Income) or EXP (Expense)\n#
AMOUNT: Positive decimal number\n\n");
        Node *current = list;
        for (int i = 0; i < *length; i++){
            // Indicate the ones that are marked for deletion, and then continue to the next
Node
            if (strcmp(current->status, "--- d") == 0){
                printf("- **%s (%ld) was removed**\n", current->desc, current->amnt);
                current = current->tail;
                continue;
            }
            // Each transaction gets saved into transaction_log.txt in the following format:
[desc] [amnt] (saved)
            else {
                fprintf(log, "%s\t%ld\t(saved)\n", current->desc, current->amnt);
                // Indicate the ones that were inserted at a specific position
                if (strcmp(current->status, "+++ i") == 0){
                    if (current->amnt >= (long)0){
                        printf("INC(insert)|%s|%ld\n", current->desc,
current->amnt);
                    }
                    else {
                        printf("EXP(insert)|%s|%ld\n", current->desc,
current->amnt);
                    }
                }
                else if (current->amnt >= (long)0){
                    printf("INC%s|%s|%ld\n", current->status, current->desc,
current->amnt);
                }
                else {
                    printf("EXP%s|%s|%ld\n", current->status, current->desc,
current->amnt);
                }
            }
            current = current->tail;
        }
    }
    fclose(log);
    clear_memory(&list);
    printf("\nDone. Exiting program.\n");
    return 0;
}

```

Explanation:

This section is responsible for saving the current session of transactions and also clearing memory that was allocated for the linked list. We first open `transaction_log.txt` in write mode using **FILE *log = fopen("../logs/transaction_log.txt", "w");** and then check if the file exists. If it doesn't, then we just clear the memory and exit the program. If the file does exist, then we begin to overwrite. If a transaction has been marked for deletion ("--- d"), then we skip to the next transaction. Otherwise, we use **fprintf(log, "%s\t%ld\t(saved)\n", current->desc, current->amnt);** to print our transaction into `transaction_log.txt` in our desired format: `desc[tab space]amnt[tab space]status`. We do this to every transaction in our current session, with every transaction getting the status "(saved)",

and then after all transactions have been saved, we clear the memory allocated for the linked list and quit the program.

Screenshots

```
trash@ThrashMachine:~/6780676-ICCS121-A2/6780676-ICCS121-A2/release$ ./fintrack
**Starting the Program (Loading Previous Transactions)**

**Original Log File (logs/transaction_log.txt) Before Running the Program:**
```
Format: TYPE|DESCRIPTION|AMOUNT
TYPE: INC (Income) or EXP (Expense)
AMOUNT: Positive decimal number

INC|Freelance Project|250.00
EXP|Phone Bill|-45.00
EXP|Electricity Bill|-100.00
```

**Program Startup Output:**
```
Welcome to your Personal Finance Tracker!

Would you like to resume your previous session? (y/n): n

Available actions:
add income [amount] [description]
add income [amount] [description] [position]
add expense [amount] [description]
add expense [amount] [description] [position]
delete [position]
print
quit
Disclaimer: [position] starts from 1

Current total: 0
Budget Status: Within Budget
```

> |
```

Current total: 0
Budget Status: Within Budget
\\

...

> add income 300 Allowance
Income added.

Current total: 300
Budget Status: Within Budget
\\

...

> add income 100 Gift
Income added.

Current total: 400
Budget Status: Within Budget
\\

...

> add expense 70 Bill
Expense added.

Current total: 330
Budget Status: Within Budget
\\

...

> add expense 90 Dinner
Expense added.

Current total: 240
Budget Status: Within Budget
\\

...

> add income 250 paycheck
Income added.

Current total: 490
Budget Status: Within Budget
\\

...

Current total: 490
Budget Status: Within Budget
\\

...

> ad income 100 paycheck

Invalid Command

Current total: 490
Budget Status: Within Budget
\\

...

> add income 10

Invalid Command

Current total: 490
Budget Status: Within Budget
\\

...

> print
[Transactions]
1. Allowance 300 (new)
2. Gift 100 (new)
3. Bill -70 (new)
4. Dinner -90 (new)
5. paycheck 250 (new)

Current total: 490
Budget Status: Within Budget
\\

...

> add expense 45 Lunch 3
Expense added at position 3.

Current total: 445
Budget Status: Within Budget
\\

...

```
Current total: 445
Budget Status: Within Budget
```

...

> delete 6
Transaction at position 6 marked for deletion.
```

```
Current total: 195
Budget Status: Within Budget
```
```

```
...

> print
[ Transactions ]
1. Allowance      300      (new)
2. Gift 100       (new)
3. Lunch          -45      +++ i
4. Bill -70       (new)
5. Dinner         -90      (new)
6. paycheck       250      --- d
```

```
Current total: 195
Budget Status: Within Budget
```
```

```
...

> quit
```

```
Saving transactions to file...
```

```
Format: TYPE|DESCRIPTION|AMOUNT
TYPE: INC (Income) or EXP (Expense)
AMOUNT: Positive decimal number
```

```
INC(new)|Allowance|300
INC(new)|Gift|100
EXP(insert)|Lunch|-45
EXP(new)|Bill|-70
EXP(new)|Dinner|-90
- **paycheck (250) was removed**
```

```
Memory cleared
```

```
Done. Exiting program.
```

```
trash@ThrashMachine:~/6780676-ICCS121-A2/6780676-ICCS121-A2/release$./fintrack
Starting the Program (Loading Previous Transactions)

Original Log File (logs/transaction_log.txt) Before Running the Program:
```
# Format: TYPE|DESCRIPTION|AMOUNT
# TYPE: INC (Income) or EXP (Expense)
# AMOUNT: Positive decimal number

INC|Freelance Project|250.00
EXP|Phone Bill|-45.00
EXP|Electricity Bill|-100.00
```

Program Startup Output:

```
Welcome to your Personal Finance Tracker!

Would you like to resume your previous session? (y/n): y

Resuming from last session...

Income added.
Income added.
Expense added.
Expense added.
Expense added.

Last session loaded.

Available actions:
add income [amount] [description]
add income [amount] [description] [position]
add expense [amount] [description]
add expense [amount] [description] [position]
delete [position]
print
quit
Disclaimer: [position] starts from 1

Current total: 195
Budget Status: Within Budget
```

```
```

```
Current total: 195
Budget Status: Within Budget
\ \
```

```
\ \
```

```
> print
[ Transactions ]
1. Allowance      300      (saved)
2. Gift 100       (saved)
3. Lunch          -45      (saved)
4. Bill -70       (saved)
5. Dinner         -90      (saved)
```

```
Current total: 195
Budget Status: Within Budget
\ \
```

```
\ \
```

```
> |
```